PROJECT BLUEPRINT / AI × COMPUTER VISION × CAD

ArchAI Studio

Multi-Agent AI for Floor-Plan Understanding, Generative Interior Design & 3D Visualization



An end-to-end research and implementation plan for a two-developer academic project, combining floor-plan computer vision, Qwen-based LangGraph agents, constraint solving, product catalogues, and a custom 2D/3D viewer.

DOCUMENT	VERSION	DATE
Technical Project Guide	1.0	November 2026

Concept and implementation blueprint · Figures are illustrative · Not a construction-ready architectural design tool

Contents

A complete overview of the product vision, AI architecture, data model, implementation plan and research evaluation.

1. Executive summary 3

2. What the application looks like 3

3. Overall system architecture 5

4. Option A — Upload an existing floor plan 6

5. Option B — Generate a floor plan from text 7

6. The shared architectural scene model 9

7. LangGraph: agents and tools 10

8. Product and materials recommendation 13

9. Geometry validation and design quality 14

10. Technical stack 15

11. Suggested repository structure 16

12. Roadmap for a team of two 17

13. Research questions and evaluation 18

14. Risks, limitations and guardrails 19

15. Possible future features (not V1) 20

16. Project deliverables 20

17. Technical references 20

V1 scope · Upload a floor plan or generate one from text. No voice input or full manual CAD drawing mode in the initial release.

1. Executive summary

ArchAI Studio is an intelligent interior-design application built **from scratch**, inspired by the layout and workflow of professional CAD software. Its primary feature is turning an architectural floor plan into a structured, editable **2D/3D interior design**, with AI-assisted furniture placement, material suggestions, budget estimation, and geometric validation.

The first version deliberately supports **two starting options only**:

Input path	What the user does	What the system produces
Option A — Upload floor plan	Upload a PDF, PNG, or JPG of an existing plan	A computer-vision reconstruction of walls, rooms, doors, and windows, followed by user review
Option B — Generate with AI	Describe an apartment/room in natural language	A candidate floor plan generated by a language model + constraint solver

Both paths converge to **the same scene format**. Specialized LangGraph agents then propose an interior layout, select catalogued furniture and materials, calculate costs, and validate the design before applying changes to the custom viewer.

What this is — and what it isn't

- **It is:** a research-oriented AI interior-design prototype with a custom web-based CAD viewer, computer vision, constraint optimization, product retrieval, and multi-agent workflows.
- **It is not:** a replacement for a licensed architect, a structural engineering tool, or a certification system for building-code compliance.
- **V1 does not include:** voice commands, a complete BricsCAD/AutoCAD clone, arbitrary multi-storey BIM, automatic guaranteed scale from a scan, or unrestricted AI-generated photorealistic 3D assets.
- **Minimal editing remains:** users must be able to correct detected walls, openings, and scale, even though manual drawing from an empty canvas is not a V1 entry path.

2. What the application looks like

The target interface uses a **professional dark CAD-style workspace**: the plan or 3D viewport in the center, contextual properties and validation results to the right, and AI controls for generating or furnishing the design. The user can switch between 2D and 3D perspectives of the **same underlying project**.

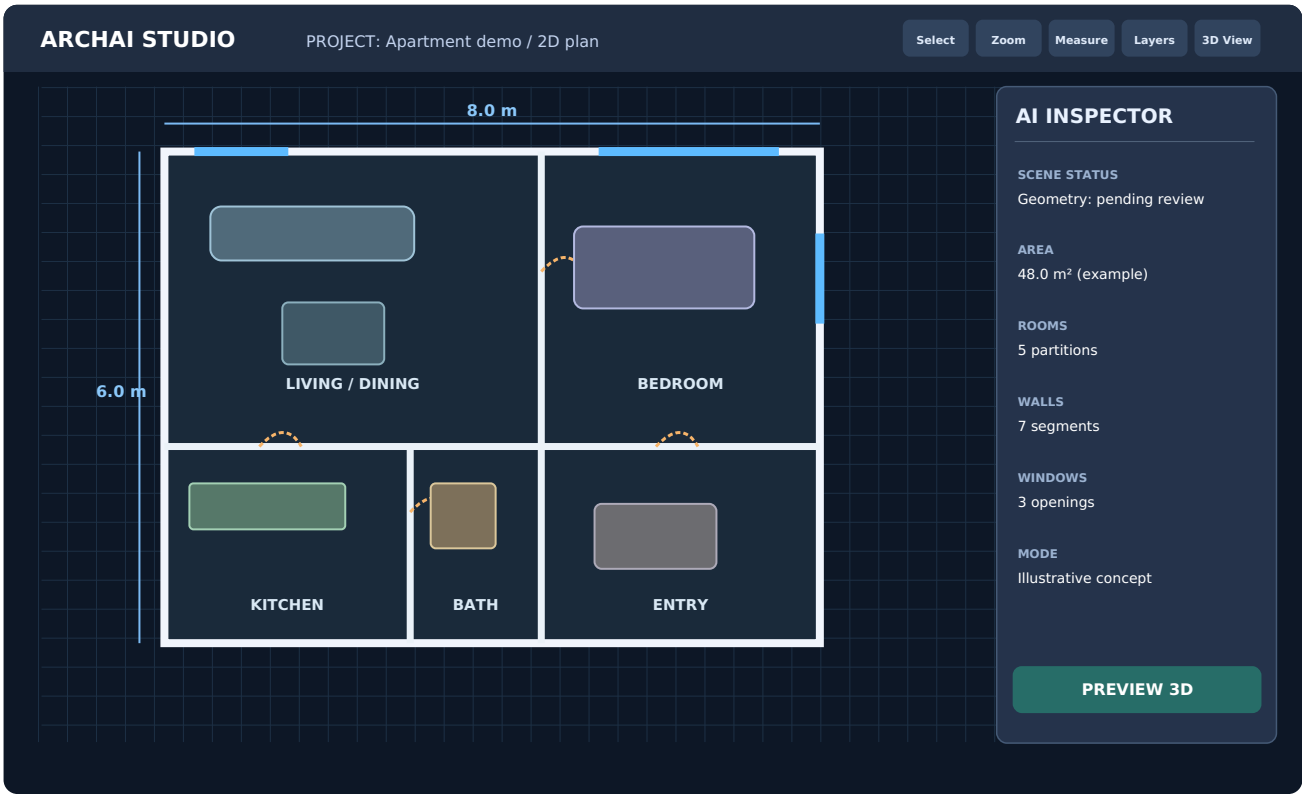


Figure 1 — Illustrative floor-plan view, not a running application. It demonstrates an editable architectural scene with room partitions, object properties, and an AI inspector.

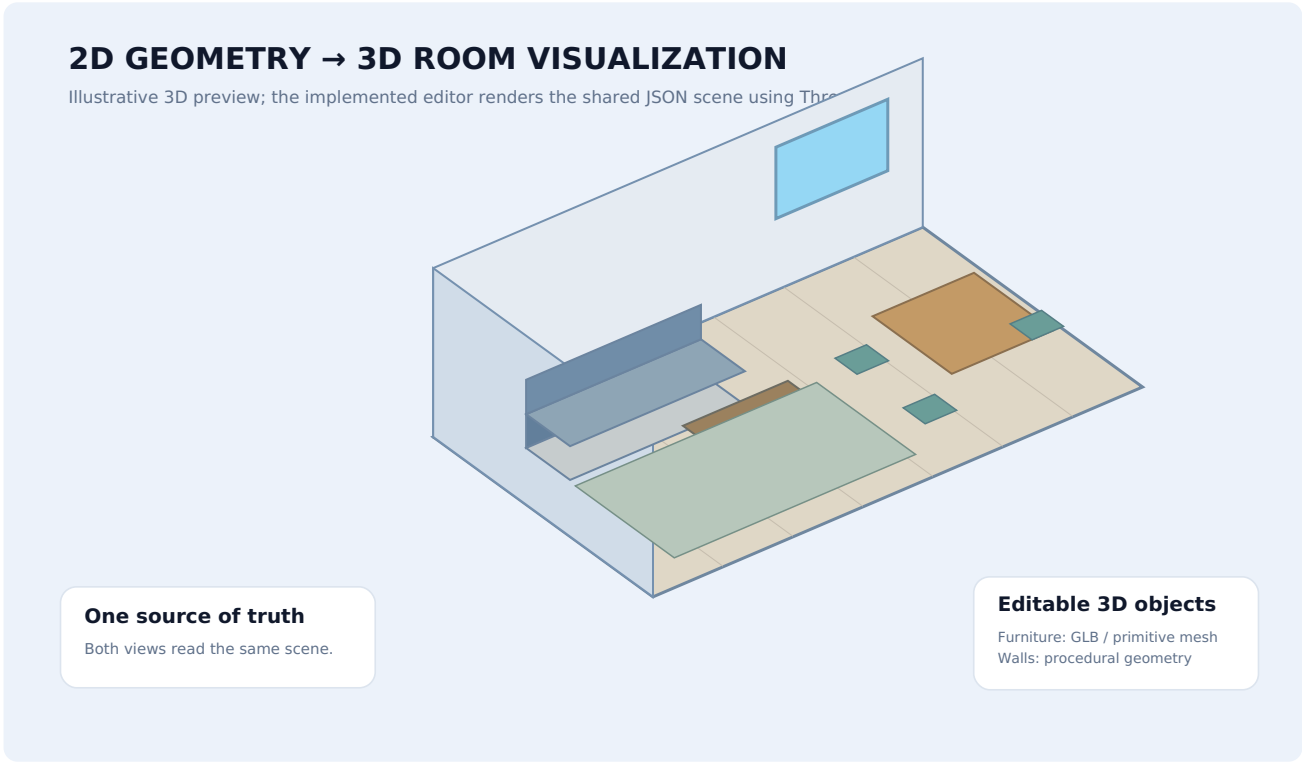


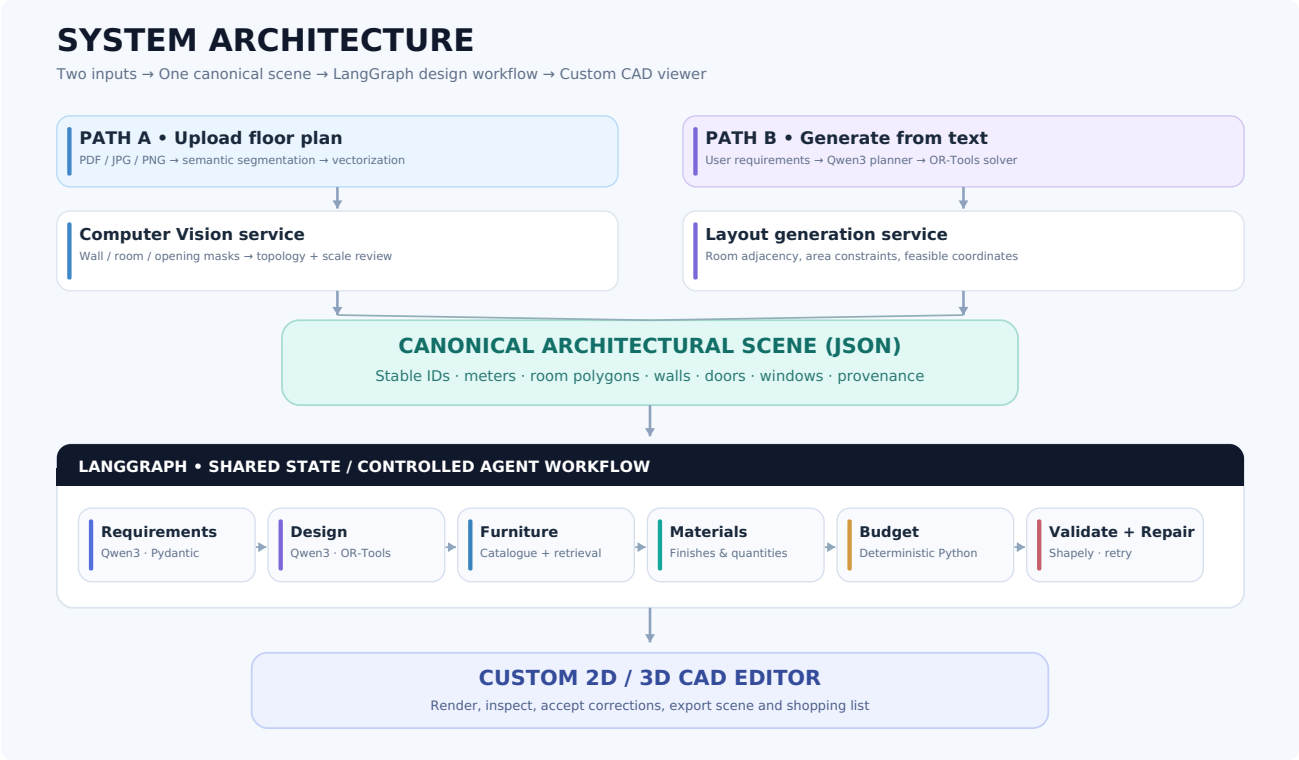
Figure 2 — Example 3D concept. The real application will build walls and floors procedurally from JSON geometry and load authorized GLB/glTF furniture assets.

User experience

1. **Start a project:** choose **Upload** or **Generate with AI**.

2. **Review the proposed plan:** confirm real dimensions, room boundaries, doors, and windows; correct extraction or generation errors.
3. **Set the design brief:** preferred style, room use, furniture categories, materials, and maximum budget in MAD.
4. **Run the design workflow:** agents propose placements and products; deterministic services validate constraints.
5. **Inspect the result:** switch between 2D/3D, review warnings, product links, approximate costs, and alternatives.
6. **Approve or reject:** the app saves an accepted version, while draft operations remain reversible.

3. Overall system architecture



The system is separated into **three engineering layers**:

Layer	Responsibility	Primary owner
Frontend CAD & 3D	Custom viewer, properties panel, object selection, scene previews, imports, approval UX	Frontend/3D developer
Perception & geometry	Plan image analysis, vectorization, layout optimization, collision and spatial checks	AI/backend developer

Layer	Responsibility	Primary owner
LangGraph orchestration	Request interpretation, design planning, furniture/material recommendations, correction workflow	AI/backend developer

Critical architectural decision: 2D, 3D, AI agents, and validation tools must read and write the **same versioned architectural scene**. The LLM must not invent different 2D and 3D structures independently.

4. Option A — Upload an existing floor plan



4.1 Supported formats

- **PNG / JPG:** raster plan images.
- **PDF:** convert pages to high-resolution images; allow the user to select the relevant page. Native vector extraction can be a later optimization.
- **Optional later:** DXF import, multipage floor plans, scanned hand-drawn plans.

4.2 Computer Vision pipeline

Step	Technical task	Suggested implementation
1. Import	Decode or rasterize the plan and preserve metadata	FastAPI upload + PyMuPDF/Pillow
2. Preprocessing	Deskew, resize, denoise, normalize and/or threshold	OpenCV
3. Perception	Recognize wall, room, door, window regions	PyTorch; U-Net-style baseline, possibly SegFormer
4. Geometric reconstruction	Skeletonize/approximate walls; form line segments and polygons	OpenCV + Shapely
5. Topology	Identify enclosed spaces, shared boundaries, door connectivity	Custom graph algorithms
6. Scale	Map pixels to metres via a **confirmed reference measurement**	Calibration UI + geometry

Step	Technical task	Suggested implementation
7. Validation	Detect unclosed walls, overlapping rooms and invalid openings	Shapely + Python rules
8. Review	Let the user correct geometry, openings and scale	Custom web editor
9. Export	Write canonical room/wall/opening objects	Pydantic + JSON

4.3 Model training direction

Use CubiCasa5K as a **research starting point**, not as a guarantee of production accuracy. It contains 5,000 annotated floor-plan images and a reference multi-task approach for plan analysis. Its original training code uses an older PyTorch environment, so a modern PyTorch implementation may be easier to maintain.

Possible research baseline: train a compact U-Net to segment `wall`, `room`, `door`, `window`, and `background` where appropriate annotation mapping supports these targets. Compare against a stronger segmentation architecture on a held-out set. Pixel-level segmentation alone is **not sufficient**; the model output still requires vectorization and topology reconstruction.

Licensing warning: CubiCasa5K is **CC BY-NC 4.0**; its restrictions matter if you later commercialize the project. Confirm permissions and licenses for every dataset, pretrained checkpoint, and training asset.

4.4 Why the review step is mandatory

A scanned floor plan may have illegible dimensions, stylized symbols, skew, occlusion, or no reliable scale. **You cannot recover exact metric sizes from pixels without a trustworthy measurement or calibrated source.** The user must confirm at least one known dimension (and preferably more) before the design is treated as metric.

Fallback for V1: upload the plan as a reference image, allow scale calibration, and permit manual correction of wall/opening geometry when computer vision is uncertain. Display a confidence/validation status rather than silently guessing.

5. Option B — Generate a floor plan from text



Example prompt:

> Create a single-storey 75 m² apartment with two bedrooms, a bathroom, a kitchen, and a living room. The living room should connect to the kitchen. Keep the main circulation clear.

5.1 Generation strategy: LLM + constraints

Do not ask Qwen to output arbitrary coordinates and assume they're geometrically correct. Split the problem into two stages:

1. **Semantic planning (LLM):** extract rooms, target areas, adjacency preferences, required openings, dimensions and constraints. Propose a room-connectivity graph or topological arrangement.
2. **Geometric solving (OR-Tools):** allocate rectangular room regions within a defined footprint using integer coordinates, non-overlap, minimum areas, adjacency and permitted opening constraints. Validate the result with Shapely.

For CP-SAT, store geometry as **integer centimetres or millimetres**, because CP-SAT models use integer decision variables; transform to metres for the JSON scene and visualization.

5.2 Requirements extracted by the LLM

```
{
  "project_type": "single_storey_apartment",
  "target_area_m2": 75,
  "rooms": [
    {"type": "living_room", "min_area_m2": 18},
    {"type": "kitchen", "min_area_m2": 8},
    {"type": "bedroom", "count": 2},
    {"type": "bathroom", "count": 1}
  ],
  "adjacency_preferences": [
    {"a": "living_room", "b": "kitchen", "preferred": true}
  ],
  "requires_user_confirmation": ["external_footprint"]
}
```

These are **example requirements**, not a verified architectural program. If exterior dimensions or major constraints are unknown, ask for them or make an explicit, reviewable assumption.

5.3 Constraint solver responsibilities

- Keep rooms **inside** the proposed footprint.
- Prevent two room polygons from overlapping.
- Respect minimum room dimensions and approximate area targets.
- Encode adjacency and accessible connections where required.
- Reserve feasible space for door openings and circulation.
- Enumerate multiple feasible layouts and score them using declared design criteria.

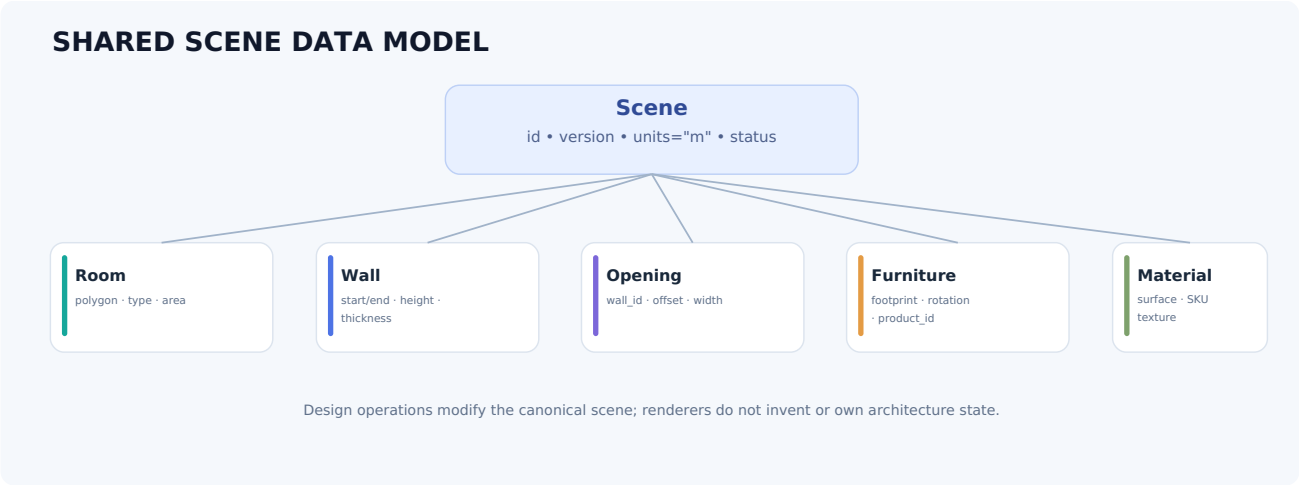
A solver can only enforce the **constraints you model**. It does not automatically understand ergonomics, architecture codes, daylight, or aesthetics.

5.4 Output

The output is a **candidate plan**: room polygons, boundaries, internal partitions, walls, openings and a report saying what passed, what was assumed, and what remains unverified. The user confirms the draft before furniture placement.

---

6. The shared architectural scene model



The **canonical JSON scene** is the contract between the backend, LangGraph agents, geometry services, and the 2D/3D editor.

Rules for the scene format

- Use **metres** in persisted scenes; convert to integer units inside CP-SAT as needed.
- Adopt a **single 2D coordinate system**: `x` and `y` represent the floor plane; define `z` as vertical only in the 3D renderer.
- Give every object a **stable ID** for editing, validation, diffing and undo/redo.
- Store doors and windows as **openings attached to a wall ID + offset**, not arbitrary independent rectangles.
- Store **rooms as polygons** and furniture as positioned instances with footprints and optional 3D asset references.
- Track the source of each object: `uploaded`, `detected`, `ai_generated`, `user_corrected`, or `catalogue`.
- Version and validate the schema with Pydantic / JSON Schema.

Example simplified scene:

```
{
  "id": "scene_001",
  "version": 1,
  "units": "m",
  "status": "draft",
  "rooms": [{
    "id": "room_01",
    "type": "living_room",
    "polygon": [[0,0],[5,0],[5,4],[0,4]]
  }],
  "walls": [{
    "id": "wall_01",
    "start": [0,0], "end": [5,0],
    "thickness": 0.2, "height": 2.8
  }],
  "openings": [{
    "id": "door_01", "type": "door",
    "wall_id": "wall_01", "offset_m": 3.7,
    "width_m": 0.9, "height_m": 2.1
  }],
  "furniture": [{
    "id": "sofa_01", "product_id": "catalog_sofa_01",
    "position": [0.7,2.6], "rotation_deg": 0,
    "width_m": 2.0, "depth_m": 0.9
  }]
}
```

This excerpt intentionally shows **only one of the four enclosing walls**. A complete 5 × 4 m scene is provided as `examples/example_scene.json`.

How the CAD viewer uses it

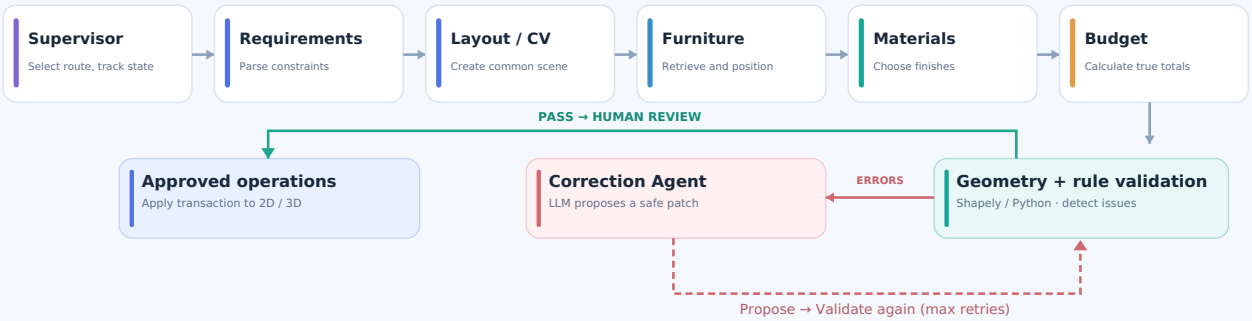
- **2D:** walls are thick line segments; rooms are polygons; doors/windows are symbols anchored to walls; furniture has editable footprints.
- **3D:** wall segments become extruded meshes; openings are cut out or represented via parametrized wall geometry; rooms become floors; furniture instances load GLB/glTF assets.
- **Agent interaction:** operations such as `create_wall`, `move_furniture`, `set_material`, and `update_opening` modify a **candidate** scene, not the rendering engine directly.

For V1, make the renderer **read-only with respect to architectural truth**: user corrections go through the same validated command service as AI edits.

---

7. LangGraph: agents and tools

LANGGRAPH AGENT WORKFLOW



All candidates remain uncommitted until the user reviews and accepts the resulting design.

LangGraph coordinates **stateful nodes** with conditional transitions. Some nodes call an LLM; others call standard Python algorithms. Calling every Python function an “agent” is unnecessary. Separate **decision-making** from **deterministic validation**.

Component	Role	Model / algorithm	Input → output
Supervisor / Router	Identify entry path, check readiness, manage workflow	Small Qwen3 LLM or deterministic router	Request → route and checklist
Requirements Agent	Extract structured intent, design style, rooms, budget and preferences	**Qwen3-8B** + Pydantic	Prompt → validated brief
Plan Understanding Service	Extract architecture from uploaded drawing	PyTorch segmentation + OpenCV	Image → wall/room/ opening candidates
Layout Agent	Propose relationships, room arrangement and constraints	**Qwen3-8B** initially; larger Qwen3 optional	Brief → topology / solver inputs
Layout Solver	Find geometrically feasible plans	**OR-Tools CP-SAT**	Constraints → room coordinates
Furniture Agent	Retrieve suitable real products, suggest placements	Qwen3-8B + SQL / embeddings	Scene + needs → product candidates
Materials Agent	Recommend finishes, calculate material requirements	Qwen3-8B + catalogue + Python	Surfaces → SKUs / quantities
Budget Service	Sum verified costs, detect over-budget designs	**Python**, not LLM	Products + quantities → cost breakdown
Geometry Validator	Test containment, collisions, openings and paths	**Shapely + Python**	Candidate scene → errors
Correction Agent	Propose bounded repairs based on concrete error codes	Qwen3-8B / larger model if needed	Errors + scene → scene operations

Component	Role	Model / algorithm	Input → output
Commit Service	Check schema, permissions, version and approval	Pydantic + transaction logic	Accepted patch → persisted scene

Why Qwen3?

Qwen3 is an open-weight language-model family and is practical for experimenting with local tool calls and structured generation. **One Qwen3-8B instance can serve several agents**, each with a different prompt, state view and tool permissions. If the layout/revision tasks are too difficult, benchmark larger models rather than upgrading blindly.

Suggested model experiments:

- **Qwen3-8B:** common LLM baseline for requirement extraction, planning and catalogue reasoning.
- **Qwen3-14B or 32B (optional):** harder planning / correction cases if hardware or hosted inference allows.
- **Qwen3-Embedding-0.6B (optional):** semantic furniture retrieval after the catalogue contains enough text to justify embeddings.
- **U-Net / SegFormer (vision research):** image segmentation; a different family from the Qwen language agents.

Do not use Qwen Image or StableGen as the floor-plan validator. Image generation can later support textures or 3D assets, but precise floor-plan validity requires geometry and constraints.

Agent workflow details

Supervisor: picks `upload` versus `generate`; requests missing scale or user requirements; routes to the appropriate perception/generation pipeline. A deterministic router is sufficient when the user explicitly clicks a start option.

Requirements: returns strict structured JSON using a Pydantic schema. Example fields: `room_count`, `must_have`, `preferences`, `budget_mad`, and `constraints`.

Layout: proposes topological relations, then asks a solver to determine coordinates. When it cannot satisfy all hard constraints, the workflow should report `infeasible` rather than produce a pretty but incorrect plan.

Furniture: gets **real catalogued products**; applies hard SQL filters for actual dimensions and price; optionally ranks style similarity using embeddings; passes candidates to placement optimization.

Materials: chooses wall/floor finishes by style and intended use; calculates quantities with verified product coverage, packaging sizes and declared waste allowances.

Budget: uses catalogue records to compute the sum in MAD. Prices and availability must have source links and `checked_at` timestamps; clearly label delivery and installation costs that are omitted or estimated.

Validation: checks geometry, product footprints, opening positions, rough circulation constraints and project-level requirements. It does not certify architecture regulations or structural safety.

Correction: proposes small **patch operations**, then runs the validator again. Cap retry count (e.g. 3) to prevent endless loops. Only commit a candidate after a successful check **and user acceptance**.

LangGraph state design (illustrative)

```
from typing import Any, TypedDict

class DesignState(TypedDict, total=False):
    project_id: str
    entry_mode: str                # upload | generate
    user_request: str
    current_scene: dict[str, Any]  # committed version
    candidate_scene: dict[str, Any]
    design_brief: dict[str, Any]
    product_candidates: list[dict]
    material_candidates: list[dict]
    budget_report: dict[str, Any]
    validation_issues: list[dict]
    retry_count: int
    status: str                    # review_needed | valid | blocked
```

An example safe command

```
{
  "type": "move_furniture",
  "scene_version": 7,
  "object_id": "sofa_01",
  "position": [0.7, 2.6],
  "rotation_deg": 90,
  "reason": "Improve access to the room entrance"
}
```

Your backend should **never execute arbitrary code generated by the LLM**. It must validate command types, numeric ranges, target IDs, current scene version and collisions. The proposal should be previewed before approval. More examples: `examples/example_operations.json`.

---

8. Product and materials recommendation

A useful differentiator is connecting a layout to **real-world furniture and materials** (for example, products found in Moroccan catalogues, subject to their access terms).

Catalogue record design

Field	Purpose
`product_id`, `merchant`, `product_url`	Exact identity and source
`name`, `category`, `description`	Search and semantic retrieval
`price_mad`, `checked_at`, `availability_status`	Grounded pricing and freshness
`width_m`, `depth_m`, `height_m`	Geometry compatibility

Field	Purpose
`finish`, `color`, `style_tags`	Style filtering
`model_3d_url`, `asset_license`	Optional visual representation
`coverage_m2_per_unit`, `pack_size`	Material quantity calculation

End-to-end selection

1. Determine available room dimensions and **hard spatial constraints**.
2. Query a curated catalogue using category, price, size and stock filters.
3. Rank eligible products by design preferences, optionally using **Qwen3-Embedding-0.6B**.
4. Calculate feasible object placements with **OR-Tools / geometric heuristics**.
5. Run collision and circulation checks.
6. Return a **product shopping list** with verified merchant links and estimated costs.

Important limitation: a retailer's product photo does not imply a downloadable 3D model. Use licensed GLB assets where available; otherwise display an approximate geometry primitive labelled as a visual proxy.

Budget example (illustrative only)

Item	Amount (MAD)
Sofa	4,800
Coffee table	1,100
Lighting	900
Flooring estimate	2,800
Paint estimate	1,300
Illustrative total	**10,900**

These numbers **are not current prices** and do not include any unlisted installation, shipping, or tax assumptions. Real application results must come from checked catalogue entries and show when they were verified.

9. Geometry validation and design quality

Separate **hard constraints** (must pass) from **soft preferences** (optimize when possible).

Validation type	Examples	Mechanism
Hard: geometry	Walls form coherent boundaries; no room overlap	Shapely geometry
Hard: openings	Door/window anchored to a compatible wall	Wall-offset checks
Hard: objects	Furniture remains inside permitted region; no overlapping footprints	Polygon tests
Hard: economics	Selected quantities and prices fit a declared budget, when mandatory	Deterministic arithmetic
Conditional	User has confirmed scale for a raster plan	Workflow state gate
Soft: circulation	Prefer accessible paths and door clearance	Configurable path / clearance scoring
Soft: style	Prefer appropriate palette and furniture combinations	Ranking / feedback
Soft: aesthetics	Prefer balanced layouts	Heuristics or expert evaluation

Never claim that a plan is structurally sound, legally compliant, or construction-ready because it passes these tests. Applicable rules depend on jurisdiction, use, and professional review. Building services, accessibility, fire safety, seismic requirements, and structural design are outside V1 validation.

10. Technical stack

Layer	Recommended technologies
Frontend	React, TypeScript, Vite, Zustand
2D viewer	React Konva or custom Canvas (editing only as needed)
3D viewer	Three.js, React Three Fiber, drei; glTF/GLB loader
API backend	FastAPI, Pydantic, Python
Agent workflow	LangGraph + LangChain model adapter
Language models	Qwen3-8B via Ollama / compatible inference API
Computer vision	PyTorch, OpenCV, Pillow, segmentation model
Geometry	Shapely, NetworkX for topology as needed
Optimization	OR-Tools CP-SAT

Layer	Recommended technologies
Catalogue DB	PostgreSQL; pgvector optional after semantic retrieval is needed
Files / scenes	Object storage or filesystem for assets; JSON/DB versions for scenes
Development	`uv`, pytest, Docker Compose, GitHub Actions

Deployment principle: keep the vision and LLM inference behind the API; the browser only needs scene JSON and render assets. Don't assume a student laptop can run a large Qwen model, CV segmentation, and high-quality 3D generation concurrently.

11. Suggested repository structure

```
archai-studio/
├── apps/
│   └── web/                    # React + TS + custom 2D / 3D interface
│       └── src/
│           ├── components/     # Toolbar, inspector, uploads
│           ├── editor2d/       # Canvas elements, selection, corrections
│           ├── editor3d/       # Scene renderers, cameras, GLB assets
│           └── state/          # Scene and UI state
├── backend/
│   ├── app/
│   │   ├── api/                # REST endpoints
│   │   ├── agents/             # Requirements, layout, furniture, materials, repair
│   │   ├── workflows/          # LangGraph graph + routing
│   │   ├── cv/                 # Segmentation, vectorization
│   │   ├── geometry/           # Shapely + topology + constraints
│   │   ├── optimization/       # OR-Tools models
│   │   ├── catalog/            # Products, pricing, material quantities
│   │   ├── schemas/            # Pydantic scene and command contracts
│   │   └── persistence/        # Project, version, asset storage
│   ├── tests/
│   └── pyproject.toml
├── data/
│   ├── catalog_sample.json
│   └── evaluation_cases/
├── assets/                     # Licensed sample GLB models and textures
└── docs/                       # Architecture, API, evaluation, licenses
```

Recommended API boundaries

Method	Endpoint	Function
`POST`	`/projects`	Create a project
`POST`	`/projects/{id}/upload-plan`	Upload a plan and start extraction
`POST`	`/projects/{id}/generate-plan`	Generate a candidate plan from requirements
`GET`	`/projects/{id}/scene`	Return latest committed scene

Method	Endpoint	Function
`POST`	`/projects/{id}/validate`	Validate current candidate
`POST`	`/projects/{id}/design-interior`	Run furniture/material/budget workflow
`POST`	`/projects/{id}/approve`	Commit reviewed operations
`GET`	`/projects/{id}/products`	Return selected catalogue products

These are **proposed interfaces**, not existing live endpoints. Prefer job IDs and progress polling for slow CV / optimization / LLM jobs; protect imports and image uploads with file-size, type and content checks.

12. Roadmap for a team of two

A realistic **target** for a research prototype is 12 weeks, with a reduced demo scope if time is short. This timeline is illustrative, not guaranteed.

Weeks	Development milestone	Deliverable
1–2	Scene schema + React editor skeleton + FastAPI	Render a saved sample room in 2D/3D
3–4	Text requirements + solver baseline	Generate a simple feasible plan from a prompt
5–6	Upload pipeline + CV baseline	Parse a clean raster floor plan into candidate geometry
7–8	User correction flow + validation engine	Confirm scale and fix geometry errors
9–10	Furniture + materials + budget workflow	Furnished room with linked product records and cost
11–12	Repair loop + testing + demo polish	End-to-end two-path demo with validation report

Team responsibilities

Developer A — frontend & 3D: React interface, 2D viewer, 3D procedural geometry, furniture asset loading, editing, selection and approval UI, scene integration.

Developer B — AI, CV & backend: floor-plan segmentation, vectorization, Qwen/LangGraph services, optimization models, validation, catalog retrieval, API, evaluation.

Shared responsibilities: JSON scene schema, API contract, integration tests, sample projects, research experiments and the final presentation.

MVP scope controls

- One storey; small apartment / single room only.
- Primarily straight, axis-aligned walls and rectangular rooms.
- Small curated catalogue (20–50 products) with verified dimensions.
- 2D upload of clean architectural plans before handling photos/sketches.
- Text generation that prefers **valid constrained layouts** over architectural novelty.
- Edits only for reviewing/correcting a proposed plan; **no blank-canvas CAD drawing workflow in V1**.

13. Research questions and evaluation

Candidate research questions

1. **Floor-plan understanding:** Does a segmentation model plus topology reconstruction yield fewer invalid wall/room structures than simple classical image processing?
2. **Constrained generation:** Does Qwen3 + OR-Tools produce a higher hard-constraint satisfaction rate than an LLM directly generating coordinates?
3. **Agentic correction:** Does deterministic validation plus a bounded repair loop improve successful design completion without increasing hallucinated operations?
4. **Grounded recommendations:** Do structured size/price filters improve furniture compatibility compared with semantic retrieval alone?

Metrics

Module	Suggested quantitative metrics
CV segmentation	Per-class IoU / Dice, door/window precision-recall
Vectorization	Wall endpoint error, polygon closure rate, room adjacency F1
Plan generation	Feasible-layout rate, required-room coverage, area/ adjacency constraint satisfaction
Furniture placement	Collision-free rate, clearance violations, successful placements
Catalogue	Correct product ID/URL rate, dimension compliance, budget compliance
Agent workflow	Valid command rate, repair success, tool failures, number of retries
UX/system	End-to-end latency, manual corrections per plan, user acceptance rate

Evaluation protocol: create fixed test splits and a small human-reviewed benchmark of plans. Compare each method with a simpler baseline. Report failures, not just best-looking rendered examples. If using CubiCasa5K, respect dataset licensing and avoid training/test leakage.

Suggested demo cases

- **Case A — Upload:** user uploads a clean 2-bedroom apartment raster plan, confirms one known dimension, corrects one misplaced door, and obtains a 3D interior preview.
- **Case B — Text:** user describes a one-room living space, selects Scandinavian style and a MAD budget, and obtains a furniture layout with a validation report.
- **Case C — Failure:** intentionally request impossible room sizes or an over-budget selection; show the solver returning `infeasible` or proposing alternatives rather than inventing success.

14. Risks, limitations and guardrails

Risk	Mitigation
CV fails on scans or unusual symbols	Confidence display + quick manual correction + constrained dataset
Wrong metric scale	Explicit measurement calibration and review gate
LLM hallucinates prices or object IDs	Retrieve from catalogue; validate against schema and real IDs
Solver cannot fit room requirements	Return infeasible conditions, request revised constraints
Furniture looks real but is not the exact product	Distinguish product SKU from generic / licensed GLB visual proxy
Agent overwrites previous work	Versioned operations, transaction previews, undo, user approval
Rendering and layout disagree	One canonical scene; automated 2D/3D consistency tests
Inference too expensive or slow	Quantization, caching, smaller models, staged workflow
Architecture/regulatory misclaims	Explicit "concept, not construction-ready" disclaimer
Copyright / commercial use conflicts	Review dataset, model, texture, CAD file and product licenses

Minimum security and privacy requirements

Use access-controlled projects, validated uploads, input-size limits, server-side file handling, no arbitrary code execution from prompts, source URL sanitization, and bounded LangGraph retries. Protect users' uploaded floor plans as sensitive project documents; define a retention/deletion policy and do not train on them without authorization.

15. Possible future features (not V1)

- Voice-controlled changes using speech recognition and TTS.
- Full manual CAD drawing mode and advanced parametric BIM.
- DXF and richer professional import/export.
- Multi-floor buildings, complex polygons, stairs, and accurate MEP modeling.
- Automatic furniture mesh generation using StableGen / Blender / other licensed tools.
- Photorealistic rendering, style transfer, VR walkthroughs.
- Professional rule review tied to a verified jurisdiction-specific regulatory corpus.

16. Project deliverables

Expected academic/software deliverables: a runnable web demo, FastAPI service, trained/evaluated CV baseline, LangGraph workflow, constraint model, scene JSON schema, 2D/3D visualization, catalog sample, geometry validator, tests, evaluation report, and a final presentation.

Success criteria for the MVP: given a clean uploaded plan **or** a sufficiently constrained text prompt, the system can create a reviewable scene, allow essential corrections, propose furniture/material choices, and clearly show geometry checks and estimated cost. A passing internal check must not be described as professional architectural approval.

17. Technical references

The following are useful starting points for implementation. APIs, model availability and licenses should be rechecked before development or commercial reuse.

1. **LangGraph** — official documentation and quickstart.
2. **Qwen3-8B** — official model card.
3. **Qwen3-Embedding-0.6B** — official model card.
4. **CubiCasa5K** — dataset and multi-task model repository and license.
5. **Google OR-Tools** — CP-SAT documentation and 2D packing implementation.
6. **Shapely** — geometry manual.
7. **Three.js** — documentation and glTF loader.
8. **React Three Fiber** — documentation.
9. **StableGen (optional future assets)** — repository.